

WCHN Cluster

The WCHN has a HPC Cluster computing facility with 302 cores and over 2TB of RAM. Matlab R2021a, FSL, Freesurfer, perl, python and C compiler all available.

To use the facility please contact fil.support@ucl.ac.uk to request and enable ssh access to holly (to submit jobs) and SFTP access to holly-host (to transfer data).

To see how busy the cluster is click [here](#). These graphs show the load average on the cluster for one day and are customisable.

HPC systems are typically used for non-interactive processes. They use a queue manager to allow you to submit multiple jobs (so they can be efficiently spread across the many processors the system has) and track the progress of those jobs. Holly uses something called Sun Grid Engine for this purpose. This is the same queue manager that UCL's HPC Legion uses, so it is easy to take a job and run it on either of the systems with minimal changes.

By it's nature, using a cluster means some use of the command line and ideally some knowledge of Linux is required. We're on a learning curve ourselves, including documentation. What follows is not comprehensive, but will hopefully help to get you started. Please get back to us with any questions or suggestions.

Use of WCHN Cluster Services

When we have enabled cluster access for you, use your normal login name and password.

Use [putty](#) or [MobaXterm](#) to SSH into holly to submit your jobs.

Use [Filezilla](#) or [WinSCP](#) to connect to holly-host to put your data on the cluster. Copy it to:
/data/holly-host/username

Charm is accessible from Holly at /data/scannernam for example /data/quattro.

Use the command qsub to submit jobs via

```
qsub -l vf=<size> -l h_vmem=<size> <scriptname>
```

Where <size> is the maximum memory size of your job. There is a default of 4GB if you do not specify anything. Before you run it, you will probably not know the maximum amount of memory that it uses. To find out, run a single (or a few jobs) specifying a memory size larger than you think you will need, e.g. 10G, and note the job number. When your job has finished run the command below on holly :

```
qacct -j <jobnumber>
```

e.g. if your job is number 28611, give the command -

```
qacct -j 28611
```

Look for the entry “maxvmem”. Round this up to the nearest GB and use that when submitting your job. e.g. if you want to run “mysubmitscript” with a maximum memory usage of 8GB, use :

```
qsub -l vf=8G -l h_vmem=8G mysubmitscript
```

BE SURE to use G for GB. Using vf=8 would request a memory size of 8 bytes! Your job will not start until the specified amount of memory is available, and will terminate if it uses more memory than specified.

If you wish you can start by submitting one of the sample scripts found in the /opt/gridengine/examples/jobs directory using the qsub commands

```
qsub /opt/gridengine/examples/jobs/simple.sh
```

or

```
qsub /opt/gridengine/examples/jobs/sleeper.sh
```

Use the qstat command to monitor the job's behaviour

```
qstat -f
```

After the job finishes executing, check your log directory /data/holly-host/username/logs for the redirected output files

The syntax of the following scripts can be used as starting points for submitting your jobs:-

Example 1

```
#!/bin/bash

# # Overly verbose script showing you how to write a script to submit
# a job to the FIL cluster. Unchanged, this will run the Matlab
# benchmark.
#
# Effectively, you will need to add the lines of this script
# to your existing script (if you have one). Simply calling
# your script from this one is likely to lead to heartach and pain
#
# Matlab must be started from your home directory.
# 'cd' to the required directory from within your matlab script
#
# lines starting "$#" are qsub options which affect how your job
# is run by the cluster.

#-----let's set some things -----

# shell to use on the cluster. This needs to be specified.
# Try using the one at the top of your existing script.

#$ -S /bin/bash
```

```
# The output from each job - the stuff you would normally see on your screen
-
# is written to files. Normally one for error messages, and one for normal
output.
# The line below joins the two so everything goes to one file.

#$ -j y

# name the job - used as the name of the output file
# NB qsub options DON'T expand input parameters e.g. $1, so don't try!

#$ -N benchtest

# To run as an array job, ** uncomment the line below (delete one '#') **
# so that it starts with: #$ -t
# As written, the array job will be run with task ID's from 1 to 100, with
steps of 2
# i.e. 1,3,5 etc.
# within your script, the variable $SGE_TASK_ID will be set to the task ID.
# Use this to decide what to do. Replace the numbers as required -
# - default step is 1. all these are OK: -t 1 , -t 35-97 , -t 103-2036:step7

##$ -t 1-100:2

# Matlab will only start properly from my home directory,
# so also added a 'cd' statement to your Matlab script
#
# If you need to use SPM functions, then add : /data/spm8
# to your Matlab path using: addpath /data/spm8
# within your Matlab script

matlab_command="bench"
/share/apps/matlab -softwareopengl -nodesktop -nosplash -nodisplay -
singleCompThread -batch $matlab_command
```

Save this example with the filename **bench.sh** and run using the following command (Note the memory requested):

```
qsub -l vf=12G -l h_vmem=12G bench.sh
```

If the job runs successfully, you will have an output file in your /data/holly-host/username/logs directory with Matlab benchmark results.

Example 2

This example script is courtesy of Peter Zeidman - a job to estimate all DCM models in a given directory.

```
#!/bin/bash

# Estimates all DCM models in a given directory
#
# Instructions:
#
# 1. Put everything in the right place. This script should go in your home
# directory (/home/username - where username is your username!), and the
# models you want to estimate should
# go into /data/holly-host/username/. It doesn't matter if the models are
# all together or
# in subfolders.
#
# 2. Count the total number of models (across all subjects) to estimate and
# enter it below (under "The total number of models..."
#
# 3. Set any other parameters below as appropriate
#
# 4. Run this script with: qsub run_cluster_dcms.sh
# Look for the output in your home directory. The estimated models will
# remain in-situ in the /data/holly-host/username/ folder.
#
# By Peter Zeidman 2013 - edited IT support - 2017

# -----
# ESSENTIAL PARAMETERS - SET BEFORE RUNNING
# -----

# Where to find the models
data_root_dir='/data/holly-host/username/models/'

# The total number of models across all subjects (written 1-n)
#$ -t 1-3

# Where to find SPM
spm_path=/data/holly-host/username/spm

# Name for this job
#$ -N dcm_estimation

# Uncomment this line ONLY for debugging
# SGE_TASK_ID=1

# -----
# IDENTIFY MODEL TO ESTIMATE
# -----

# List all models for all subjects
models=($(find $data_root_dir -name "DCM_*" | sort -n))

# Identify model to run here
```

```
model_to_run=${models[$SGE_TASK_ID-1]}

# Report
total_models=${#models[@]}
printf 'Task %d: Found %d models, estimating %s\n' $SGE_TASK_ID
$total_models $model_to_run

# -----
# CLUSTER JOB OPTIONS
# -----

# shell to use on the cluster

#$ -S /bin/sh

# The output from each job - the stuff you would normally see on your screen
-
# is written to files. Normally one for error messages, and one for normal
output.
# The line below joins the two so everything goes to one file.

#$ -j y

# Run Matlab

matlab_command="addpath('$spm_path');spm_dcm_estimate('$model_to_run');"
/share/apps/matlab -nojvm -nodesktop -nosplash -nodisplay -singleCompThread
-batch $matlab_command
```

It is preferable to use qsub to run a script because it can include options, designated by lines beginning: # \$

Remember that your script should start by defining the shell: # ! and also define the shell for the cluster job (see below). # \$ options do NOT expand input parameters. \$1 literally means “\$1”

\$ options include:

- S <shell>

- j y join standard and error output into one file

- N jobname

- t n-m:s array processing - run with \$SGE_TASK_ID = n to m step s

SGE adds the following shell variables - \$HOME, \$USER, \$JOB_ID, \$HOSTNAME, \$JOBNAME and for array processing, \$SGE_TASK_ID, \$SGE_TASK_FIRST, \$SGE_TASK_LAST and \$SGE_TASK_STEP.

Matlab, and SPM

The default version of Matlab on the cluster, currently R2021a, can be run using

```
/share/apps/matlab -nojvm -nodesktop -nosplash -nodisplay -singleCompThread
```

When running matlab ensure that you are running it in your home directory. Within your matlab script change directory to wherever it is you need to be.

If SPM8 or SPM12 is required, put a copy in /data/holly-host/username folder and add it to the path within matlab using the addpath as part of your Matlab script.

Submitting Jobs

A useful introduction to submitting jobs for a cluster can be found at - <https://wikis.utexas.edu/display/CCBB/sge-tutorial>.

It is better for all users of the cluster facility if you can run many short jobs e.g. 48 x 1-hour jobs are a better choice than 1 x 48-hour job. Array jobs can also be run. Please see the relevant commented lines in the example scripts.

For a cluster command line reference please follow this [pdf](#)

Cluster Etiquette

The cluster is by its nature a shared resource.

To see how busy the cluster is click [here](#). These graphs show the load average on the cluster for one day and are customisable.

It is perfectly possible for a single user to submit sufficient jobs to fill the queue for years of processing, which is clearly unreasonable to other users.

Please discuss your plans with the IT team if you are planning to submit jobs that the entire cluster will take longer than 24 hours to execute. Remember that the total run time for what you submit depends both on how many jobs you submit, as well as how long each individual job runs for.

If practicable, try and size individual jobs so that they run for an hour or two. Longer running jobs raise the likelihood of large numbers of nodes standing idle while you wait for some tail end jobs to finish.

Generally the cluster is managed so that the first job submitted will run first, but at times we will override this to allow people who have submitted small jobs turn them around in a timely fashion when the cluster is saturated and they would otherwise be facing a lengthy wait.

Specifying Memory Requirements

Submit your jobs with only the memory requirements they need. When you submit a job the memory you specify is effectively reserved for those tasks. If your task does not use the memory it cannot be used for any other task. This unnecessarily limits the number of jobs that can run at once, slowing down the cluster for you and every other user.

e.g. if a node has 46GB of memory and you specify 6GB for your job, only 7 tasks will fit on it. If you can reduce that to 5GB, then 9 jobs will run on the node, almost 30% more jobs.

Finding the Amount of Memory Required

Run a few jobs, or an array job, and use the scripts below ("Useful Scripts on Holly") to find the amount of memory used. Specify only a little more than the maximum.

If the memory requirements vary considerably e.g.increase with the task number, or otherwise can be split into groups, consider splitting it into 2 or more jobs, each with different memory requirements.

If only the occasional task has high memory requirements, consider specifying sufficient for most of the tasks to run and re-run the failed ones with the required larger memory specified.

NB If your jobs have an exit_status of 137, it probably ran out of memory. Check the status by using `qacct -j <jobnumber>`

e.g. `qacct -j 12346`

From:

<http://intranet.fil.ion.ucl.ac.uk/wiki/it/> - **WCHN Computing Wiki**

Permanent link:

<http://intranet.fil.ion.ucl.ac.uk/wiki/it/doku.php?id=cluster>

Last update: **2023/03/29 11:41**

